

ABC454 G - Mode in the Subtree 题解

题意概括

给定一棵以 1 为根的树，每个点有一个颜色。

对每个点 v ，考虑它整棵子树中各种颜色出现的次数：

- 记最大出现次数为 m_v
- 记达到这个最大次数的颜色种数为 k_v

题目并不要求把每个 m_v ， k_v 都输出出来，而是要求最终的：

$$\left(\sum_{v=1}^N (m_v \oplus v) \times (k_v \oplus v) \right) \bmod 998244353$$

解题思路

第一步：先把树和颜色生成出来

题目输入里没有直接给出整棵树和全部颜色，而是给了生成规则。先严格按伪代码把：

- $p_2 \dots p_N$
- $c_1 \dots c_N$

重建出来。

由于题目保证 $p_i < i$ ，所以父节点编号总是比子节点小。

第二步：需要维护“某棵子树里的颜色频率”

如果直接对每个点都单独统计整棵子树的颜色次数，复杂度会很高。

这题的标准做法是 DSU on tree (也叫 sack)：

1. 先求出每个点的子树大小 $sz[v]$ ，并找到重儿子 $heavy[v]$
2. 处理一个点 v 时：
 - 先递归处理所有轻儿子，并在处理完后把它们的统计信息清空
 - 再处理重儿子，并保留它的统计信息
 - 然后把所有轻儿子的整棵子树重新加回当前统计结构
 - 最后再把 v 自己加进去

这样做的好处是：

- 重儿子的统计结构会一直被复用；
- 轻儿子的节点只会在“被重新加回”时进入若干次；
- 总复杂度是 $O(N \log N)$ 。

第三步：当前统计结构里维护什么

当前处理到某个点 v 时，统计结构中正好保存着 v 的整棵子树。

我们维护：

- `cnt[color]`：当前颜色出现了多少次
- `max_freq`：当前最大的出现次数
- `num_max`：有多少种颜色达到了 `max_freq`

每加入一个颜色 `col` 时：

1. `cnt[col]++`
2. 如果新次数比 `max_freq` 更大：
 - `max_freq = cnt[col]`
 - `num_max = 1`
3. 如果新次数恰好等于 `max_freq`：
 - `num_max++`

因为在保留统计结构的阶段我们只会做“加入”，不会做任意删除，所以这样维护是很方便的。

第四步：如何清空

当某个子树不需要保留时，我们要把当前统计结构恢复为空。

这里用一个 `used_colors` 数组记录：

- 哪些颜色当前计数从 0 变成过正数

这样清空时只需要把这些颜色的 `cnt` 重置为 0，并把：

- `max_freq = 0`
- `num_max = 0`

即可，不需要把整张颜色表全扫一遍。

第五步：为什么可以非递归实现

数据范围很大，树可能退化成链，递归会爆栈。

但本题有一个很好的性质：

- 对任意边 $p_i \rightarrow i$, 都有 $p_i < i$

因此:

- 用显式栈做一次 DFS, 可以得到 Euler 序 $tin / tout$
- 再按 Euler 序倒着处理, 就能在线性时间里求出子树大小和重儿子
- DSU on tree 本身也可以用显式栈模拟递归状态

这样就避开了系统递归栈限制。

正确性说明

先看 DSU on tree 的统计内容。

处理节点 v 时:

- 重儿子的统计结构会被保留下来;
- 所有轻儿子的整棵子树会被重新加入;
- 最后再加入 v 自己。

因此, 在计算答案的那一刻, 统计结构中恰好包含且只包含 v 子树中的所有节点。

于是:

- $cnt[color]$ 正是颜色 $color$ 在 v 子树中的出现次数;
- max_freq 正是所有颜色频率中的最大值 m_v ;
- num_max 正是达到这个最大值的颜色种数 k_v 。

再看复杂度。

对于每个节点, 它如果属于某个轻子树, 就只会在沿着祖先向上时被重新加入若干次; 而每次重新加入之后, 对应的轻子树规模至少翻倍, 因此每个节点被重新加入的次数是 $O(\log N)$ 的。

所以总加入次数是 $O(N \log N)$, 每次加入只做常数工作, 算法整体正确且复杂度可接受。

复杂度

- 时间复杂度: $O(N \log N)$
- 空间复杂度: $O(N)$

参考实现

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
```

```

const ll MOD = 998244353;
const ll MASK = (1LL << 31);

struct Frame{
    int v;
    int edge;
    unsigned char state;
    bool keep;
};

int main(){

    // 先按题目给的伪代码重建整棵树和所有颜色
    int n, M, F;
    ll seed;
    cin >> n >> seed >> M >> F;

    vector<int> head(n + 1, -1);
    vector<int> to(n), nxt(n);
    vector<int> parent(n + 1, 0), color(n + 1, 0);

    ll state = seed;
    int edge_cnt = 0;

    // 生成父节点数组，同时顺手建树
    for(int i=2; i<=n; i++){
        int p;
        if(i <= M){
            cin >> p;
        } else {
            p = (int)(state % (i - 1)) + 1;
            state = (state * 1103515245 + 12345) % MASK;
        }

        parent[i] = p;
        to[edge_cnt] = i;
        nxt[edge_cnt] = head[p];
        head[p] = edge_cnt++;
    }
}

```

// 再生成每个点的颜色

```
for(int i=1; i<=n; i++){
    if(i <= M){
        cin >> color[i];
    } else {
        color[i] = (int)(state % F) + 1;
        state = (state * 1103515245 + 12345) % MASK;
    }
}
```

// 用显式栈跑 DFS，求每个点的 Euler 序区间，避免递归爆栈

```
vector<int> tin(n + 1), tout(n + 1), euler(n + 1), iter(n + 1);
vector<int> stk;
stk.reserve(n);
```

```
int timer = 0;
tin[1] = ++timer;
euler[timer] = 1;
iter[1] = head[1];
stk.push_back(1);

while(!stk.empty()){
    int v = stk.back();
    if(iter[v] != -1){
        int e = iter[v];
        iter[v] = nxt[e];
        int u = to[e];

        tin[u] = ++timer;
        euler[timer] = u;
        iter[u] = head[u];
        stk.push_back(u);
    } else {
        tout[v] = timer;
        stk.pop_back();
    }
}
```

// 按 Euler 序倒着推，求子树大小和重儿子

```
vector<int> sz(n + 1, 0), heavy(n + 1, 0);
for(int idx=n; idx>=1; idx--){
```

```

    int v = euler[idx];
    sz[v] = 1;
    heavy[v] = 0;

    for(int e=head[v]; e!=-1; e=nxt[e]){
        int u = to[e];
        sz[v] += sz[u];
        if(sz[u] > sz[heavy[v]]){
            heavy[v] = u;
        }
    }
}

vector<int> cnt(n + 1, 0), used_colors;
used_colors.reserve(n);

// max_freq 是当前统计结构中的最大出现次数
// num_max 是达到这个最大次数的颜色种数
int max_freq = 0;
int num_max = 0;

// 把一种颜色加入当前统计结构，并同步维护众数信息
auto add_color = [&](int col){
    if(cnt[col] == 0){
        used_colors.push_back(col);
    }
    cnt[col]++;

    if(cnt[col] > max_freq){
        max_freq = cnt[col];
        num_max = 1;
    } else if(cnt[col] == max_freq){
        num_max++;
    }
};

// 把 Euler 序上的一个连续区间整段加入统计结构
auto add_range = [&](int l, int r){
    for(int i=l; i<=r; i++){
        add_color(color[euler[i]]);
    }
}

```

```

};

// 当前这轮不保留时，只清理真正用到过的颜色
auto clear_all = [&]() {
    for (int col : used_colors) {
        cnt[col] = 0;
    }
    used_colors.clear();
    max_freq = 0;
    num_max = 0;
};

ll answer = 0;
vector<Frame> dfs_stack;
dfs_stack.reserve(n);
dfs_stack.push_back({1, head[1], 0, false});

// 用显式栈模拟 DSU on tree:
// 先做轻儿子，再做重儿子，最后把轻儿子重新加回
while (!dfs_stack.empty()) {
    Frame &cur = dfs_stack.back();
    int v = cur.v;

    if (cur.state == 0) {
        cur.edge = head[v];
        cur.state = 1;
        continue;
    }

    if (cur.state == 1) {
        while (cur.edge != -1 && to[cur.edge] == heavy[v]) {
            cur.edge = nxt[cur.edge];
        }

        if (cur.edge != -1) {
            int u = to[cur.edge];
            cur.edge = nxt[cur.edge];
            dfs_stack.push_back({u, head[u], 0, false});
            continue;
        }
    }
}

```

```

        cur.state = 2;
        continue;
    }

    // 轻儿子都算完并清空后, 开始处理重儿子
    if(cur.state == 2){
        cur.state = 3;
        if(heavy[v] != 0){
            int u = heavy[v];
            dfs_stack.push_back({u, head[u], 0, true});
            continue;
        }
        continue;
    }

    // 当前重儿子的统计结构已经保留着, 再把所有轻儿子整段加回
    for(int e=head[v]; e!=-1; e=nxt[e]){
        int u = to[e];
        if(u == heavy[v]){
            continue;
        }
        add_range(tin[u], tout[u]);
    }

    // 最后把当前点自己加入, 这时统计结构恰好就是 v 的整棵子树
    add_color(color[v]);

    // 此时 max_freq 和 num_max 就分别是 m_v、k_v
    answer = (answer + 1LL * (max_freq ^ v) % MOD * (num_max ^ v)) % MOD;

    bool keep = cur.keep;
    dfs_stack.pop_back();
    if(!keep){
        // 轻子树计算结束后不保留, 恢复为空结构
        clear_all();
    }
}

cout << answer % MOD << '\n';

```

```
return 0;
```

```
}
```

